

Here is the complete **9-Day Linux & IT Operations Lab** formatted cleanly into structured Markdown, ready to be saved as a Word document (.docx) or opened directly in Microsoft Word.

9-Day Linux & IT Operations Lab

Day 1: Linux Administration & Identity

Task 1: Employee onboarding

- **Setup:**
 - 1 Ubuntu Server VM.
 - No existing data required.
 - Create a fictional employee list:
 - **Developers:** ahmed, khalid, sara
 - **Support:** mohammed, nora
 - **Management:** faisal
 - **What to do:**
 - Create all users.
 - Give each user a home directory.
 - Configure their login shell.
 - Set initial passwords.
 - Create the developers, support, and management groups.
 - Assign employees to their correct groups.
 - Verify users with: `id username, groups username, getent passwd username, getent group groupname`.
 - **Tools might needed:** `useradd, usermod, passwd, id, groups, getent, /etc/passwd, /etc/shadow, /etc/group`.
 - **Online resources on how to approach:** `man useradd, man usermod, man passwd`, [Linux Journey - Users and Groups](#).

Task 2: Department access

- **Setup:**

- Continue using the Ubuntu VM from Task 1.
- The users and groups created in Task 1 must already exist.
- Create:
 - /company
 - /company/development
 - /company/support
 - /company/management
- **What to do:**
 - Make each department directory owned by its appropriate group.
 - Developers should have access to development resources.
 - Support should have access to support resources.
 - Management should have access to management resources.
 - Prevent unrelated users from accessing department directories.
 - Create /company/shared for resources that multiple departments are allowed to use.
 - Test access by switching between users.
 - **Tools might needed:** chmod, chown, chgrp, su, sudo, ls -l, id.
 - **Online resources on how to approach:** man chmod, man chown, man chgrp, [Ubuntu File Permissions](#).

Task 3: Employee leaves the company

- **Setup:**
 - Continue from Task 2.
 - Choose sara as the employee who has left the company.
 - Put some fictional files inside Sara's home directory first:
 - /home/sara/project-notes.txt
 - /home/sara/client-data.txt
 - /home/sara/old-project/
- **What to do:**
 - Disable Sara's login.
 - Remove unnecessary group memberships.

- Preserve her home directory.
- Archive her data (e.g., `/home/archives/sara-YYYY-MM-DD.tar.gz`).
- Verify she can no longer authenticate.
- Verify IT can still access her archived data.
- **Tools might needed:** `passwd`, `l`, `usermod`, `id`, `groups`, `tar`, `find`.
- **Online resources on how to approach:** `man passwd`, `man usermod`, [GNU tar manual](#).

Final Task of Day 1: New company branch onboarding

- **Setup:** 1 Ubuntu Server VM. Use the users/groups created during Tasks 1–3. Prepare a new fictional employee list containing 8 employees across Development, Support, and Management (one employee marked as having left).
 - **Tools needed:** Ubuntu Server, VirtualBox / VMware, `useradd`/`usermod`, `passwd`, `groups`, `chmod`/`chown`, `tar`, `sudo`.
 - **Final task:** Act as IT administrator for a new company branch. Create accounts, configure groups, set up department directories, enforce permissions, create shared storage, verify access, offboard the departing employee, and produce a short administration report.
-

Day 2: Services, Processes & System Administration

Task 1: Service management

- **Setup:** 1 Ubuntu Server VM. Install Nginx (`sudo apt install nginx`).
- **What to do:** Practice `systemctl` commands (`status`, `start`, `stop`, `restart`, `enable`, `disable`). Understand the states: Running, Stopped, Enabled, Disabled, Failed. Test behavior across reboots.
- **Tools might needed:** `systemctl`, `nginx`, `journalctl`.

Task 2: Process incident

- **Setup:** Install `stress-ng` (`sudo apt install stress-ng`). Run a controlled CPU workload: `stress-ng --cpu 1 --timeout 60s`.
- **What to do:** Use `top`, `htop`, or `ps` to identify PID, process owner, CPU usage, and name. Practice terminating the process safely.
- **Tools might needed:** `ps`, `top`, `htop`, `pgrep`, `kill`, `stress-ng`.

Task 3: Log investigation

- **Setup:** Backup Nginx config (`sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.backup`). Deliberately introduce a syntax error.
- **What to do:** Run `sudo nginx -t` and `sudo systemctl restart nginx`. Investigate using `journalctl -u nginx`. Restore configuration and verify.
- **Tools might needed:** `nginx -t`, `systemctl`, `journalctl`, `curl`.

Final Task of Day 2: Production web-server incident

- **Final task:** Resolve an internal website outage. Diagnose whether Nginx is running, check service status, test the website, inspect Nginx and system logs, identify the root cause, restore service, verify functionality, and write an incident report (Incident, Evidence, Root cause, Fix, Verification).
-

Day 3: Networking & Remote Server Administration

Task 1: Network investigation

- **Setup:** 1 Ubuntu Server VM (NAT/bridged networking).
- **What to do:** Determine IP address, network interface, default gateway, DNS configuration, and listening ports using `ip addr`, `ip route`, `ss -tulpn`, `ping`, and `curl`.
- **Tools might needed:** `ip`, `ss`, `ping`, `curl`, `hostname`, `resolvectl`.

Task 2: DNS troubleshooting

- **Setup:** Deliberately configure a temporary invalid DNS server (take a snapshot first).
- **What to do:** Compare `ping 1.1.1.1` with `ping example.com`. Investigate using `dig`, `nslookup`, and `resolvectl status`. Restore working DNS.
- **Tools might needed:** `dig`, `nslookup`, `resolvectl`, `ping`, `curl`.

Task 3: SSH administration

- **Setup:** 1 Ubuntu Server VM and a client (host PC or second VM).
- **What to do:** Create user `itadmin`, generate an SSH key (`ssh-keygen`), configure key authentication, connect remotely, and inspect SSH logs.
- **Tools might needed:** `ssh`, `ssh-keygen`, `sshd`, `journalctl`, `/etc/ssh/sshd_config`.

Final Task of Day 3: Remote administration server

- **Final task:** Prepare the server for secure remote IT administration by configuring hostname, network verification, SSH key-only access, administrative user creation, and troubleshooting a broken SSH/DNS config.
-

Day 4: Web Server, Storage & Backups

Task 1: Deploy a website

- **Setup:** Create `/var/www/company/index.html` with a company page. Configure Nginx to serve it. Test with `curl http://localhost`.

Task 2: Disk investigation

- **Setup:** Create safe test data in `/opt/lab-data/` using `fallocate`.
- **What to do:** Investigate disk usage using `df -h`, `du -sh`, and `lsblk`. Find space consumers and remove test data.

Task 3: Backup and restore

- **Setup:** Create `/var/backups/company`.
- **What to do:** Back up `/var/www/company`, `/etc/nginx`, and `/company` using `tar` or `rsync`. Delete a test file, verify deletion, restore from backup, and verify functionality.

Final Task of Day 4: Small company web server

- **Final task:** Operate the server as a production web server, handle storage monitoring, perform backups, simulate data loss, restore files, and document the backup/restoration workflow.
-

Day 5: Docker & Container Operations

Task 1: Container deployment

- **Setup:** Install Docker. Run Nginx: `docker run -d --name company-web -p 8080:80 nginx`.
- **What to do:** Inspect containers (`docker ps`, `docker logs`, `docker inspect`, `docker exec`). Stop and recreate.

Task 2: Persistent database data

- **Setup:** Run PostgreSQL with a named volume. Create sample tables (`employees`, `tickets`, `assets`).
- **What to do:** Stop and remove the container, launch a new container attaching the same volume, and verify data persistence.

Task 3: Container troubleshooting

- **Setup:** Break a disposable container (wrong environment variable, port, or command).
- **What to do:** Investigate using logs, inspect, and stats to fix the container.

Final Task of Day 5: Three-tier container deployment

- **Final task:** Build a decoupled architecture (Browser $\rightarrow$ Nginx $\rightarrow$ FastAPI $\rightarrow$ PostgreSQL) using separate containers, persistent volumes, container networks, environment variables, and health policies.
-

Day 6: Docker Compose & Application Infrastructure

Task 1: Docker Compose

- **Setup:** Create `docker-compose.yml` defining web, app, and database services. Manage with `docker compose up -d`, `ps`, `logs`, and `down`.

Task 2: Configuration management

- **Setup:** Move configuration variables (`DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`) into a `.env` file. Exclude it using `.gitignore`.

Task 3: Container networking

- **Setup:** Change `DB_HOST` to an invalid service name in Compose. Investigate connection failures using `docker network ls` and `docker exec`, then resolve the issue.

Final Task of Day 6: Internal application deployment

- **Final task:** Fully deploy the internal application stack via Docker Compose with health checks, environment variables, restart policies, and troubleshoot a simulated database connectivity failure.
-

Day 7: Linux Security & Access Management

Task 1: SSH hardening

- **Setup:** Take a VM snapshot. Disable root SSH login, disable password authentication (require keys), and restrict access. Validate with `sudo sshd -t`.

Task 2: Firewall

- **Setup:** Install/configure UFW (ufw). Allow only required ports (SSH: 22, HTTP: 80, HTTPS: 443). Verify blocked ports.

Task 3: Investigate suspicious login activity

- **Setup:** Generate failed SSH attempts locally by typing incorrect passwords.
- **What to do:** Investigate using `last`, `lastb`, `who`, `w`, and `journalctl` to identify targeted accounts, active sessions, and network connections.

Final Task of Day 7: SSH security incident

- **Final task:** Respond to a simulated brute-force attack. Investigate auth logs, check active sessions, harden SSH, apply UFW firewall rules, and document the security incident.
-

Day 8: Monitoring, Automation & Troubleshooting

Task 1: Monitoring

- **Setup:** Establish baseline system metrics (CPU, RAM, Disk, Network, Processes, Containers) using `top`, `htop`, `free`, `df`, `ss`, and `docker stats` under load.

Task 2: Automated backups

- **Setup:** Create `backup-company.sh` to timestamp, compress, and back up company data to `/var/backups/company`, pruning old backups. Schedule via `cron`.

Task 3: Troubleshooting challenge

- **Setup:** Create a randomized hidden failure (syntax error, bad DB host, stopped container, or high disk usage).
- **What to do:** Diagnose using symptoms alone, locate root cause, fix, and verify.

Final Task of Day 8: IT operations monitoring and automation

- **Final task:** Build an automated IT operations routine and generate an operational report covering uptime, resource utilization, services, containers, and backup status.

Day 9: Full Real-World IT Operations Simulation

Task 1: Employee management incident

- **Scenario:** Act on a real HR/IT ticket to onboard 3 developers, 2 support staff, and 1 manager with correct directories and group permissions, while offboarding a departing employee and archiving their data.

Task 2: Production application incident

- **Scenario:** The application is reported down due to a deleted PostgreSQL container. Investigate via Docker logs, inspect volumes, restore PostgreSQL using the existing persistent volume, and verify data integrity.

Task 3: Security and capacity incident

- **Scenario:** Investigate repeated SSH authentication failures using `journalctl/lastb` and resolve unexpected disk usage using `df/du/find` without impacting core services.

Final Task of Day 9: Build and operate infrastructure for a 20-person software company

- **Final Scenario:** Step into a junior IT/Linux administrator role for a 20-person software company operating across 6 comprehensive phases:
 1. **Identity:** Users, groups, directories, ACLs, and offboarding.
 2. **Application Infrastructure:** Nginx $\rightarrow$ FastAPI $\rightarrow$ PostgreSQL via Docker Compose.
 3. **Server Security:** SSH keys, disabled root login, UFW firewall rules.
 4. **Backup & Restore:** Automated backup script, testing, and restoration verification.
 5. **Incident Response (App Outage):** Container debugging and volume restoration.
 6. **Incident Response (Security):** Log analysis of authentication failures.
 7. **Incident Response (Capacity):** Disk space investigation and cleanup.

Example Incident / Admin Record Format

Keep a log entry in this format for every task completed throughout the lab:

Date: YYYY-MM-DD

Task: PostgreSQL container failure

Environment: Ubuntu Server / Docker Compose

Problem: Application could not connect to database.

Symptoms: HTTP 500 responses from API.

Commands/tools used: `docker compose ps`, `docker logs`, `docker volume ls`

Evidence found: `docker compose ps` showed PostgreSQL stopped; container startup issue in logs.

Root cause: PostgreSQL container was removed during the incident.

Fix: Recreated PostgreSQL using the existing persistent volume.

Verification: Existing database records were still present; FastAPI successfully connected.

What I learned: Container lifecycle and persistent storage are separate entities.